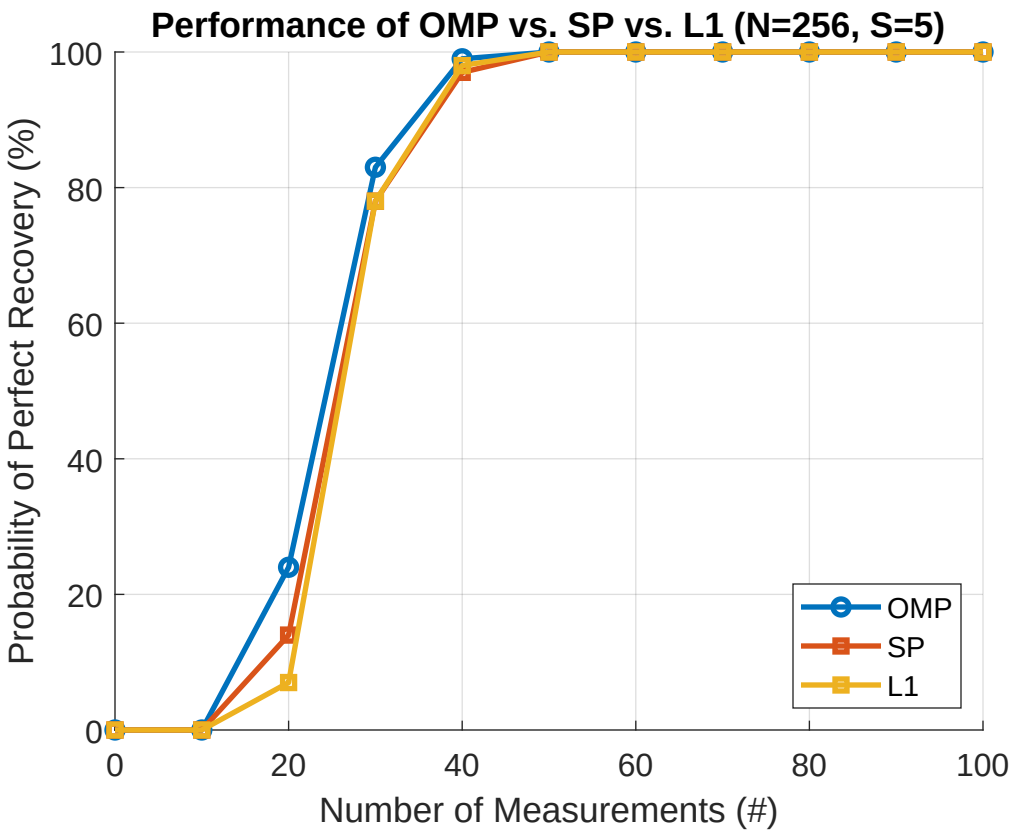
Performance of OMP vs. SP vs. L1 (N=256, S=5)
Probability of Perfect Recovery (%)
Number of Measurements (#)
0
20
40
60
80
100
OMP
SP
L1

The l1 solution seems to get largely the same performance as the SP algorithm, but it seemed noticeably slower when I timed it.

```
clearvars; close all; clc;

%% Problem 1
N = 256; S = 5;

tol = 10e-6;
num_meas = 0:10:100;
n_iter = 100;

count_sp = zeros(size(num_meas));
count_omp = zeros(size(num_meas));
count_l1 = zeros(size(num_meas));

for i = 1:numel(num_meas)
    M = num_meas(i);

    for j = 1:n_iter
        x=zeros(N,1); q=randsample(1:N,S); x(q)=randn(S,1);
        A=randn(M, N); A=orth(A')';

        y = A*x;

        x_SP = csutils.SOLVE_SP(A,y,S);
        error_SP = norm(x_SP-x, 2);

        x_OMP = csutils.SOLVE_OMP(A,y,S);
        error_OMP = norm(x_OMP-x, 2);

        x_L1 = csutils.SOLVE_L1(A,y);
        error_l1 = norm(x_L1-x, 2);

        count_sp(i)  = count_sp(i)  + (error_SP<tol);
        count_omp(i) = count_omp(i) + (error_OMP<tol);
        count_l1(i)  = count_l1(i) + (error_l1<tol);
    end
end
count_sp  = (count_sp./n_iter)*100;
count_omp = (count_omp./n_iter)*100;
count_l1  = (count_l1./n_iter)*100;


figure;
hold on;
plot(num_meas, count_omp, "-o", "LineWidth", 2, "DisplayName", "OMP");
plot(num_meas, count_sp, "-s", "LineWidth", 2, "DisplayName", "SP");
plot(num_meas, count_l1, "-s", "LineWidth", 2, "DisplayName", "L1");
hold off;
grid on;
xlabel("Number of Measurements (#)");
ylabel("Probability of Perfect Recovery (%)");
title(sprintf("Performance of OMP vs. SP vs. L1 (N=%d, S=%d)", N, S));
legend("Location", "SouthEast");
set(gca, "FontSize", 12);
```

```
function [x_hat, error] = SOLVE_SP(A, y, S, tol)
% Subspace Pursuit (SP) algorithm to recover a sparse signal.
%
% Inputs:
%   A     - Sensing matrix (M x N)
%   y     - Measurement vector (M x C)
%   S     - Sparsity level (number of non-zero elements in x)
%   tol   - Optional tolerance to stop iterating
%
% Output:
%   x_hat - Reconstructed sparse signal (N x C)
%   error - Error between A*x_hat and y
    arguments
        A   {mustBeNumeric}
        y   {mustBeNumeric, mustHaveSameRows(A,y)}
        S   {mustBeInteger, mustBePositive}
        tol {mustBeReal, mustBePositive} = 1e-15
    end

    [~, N] = size(A); % Get length of signal x
    [~, C] = size(y); % Get number of signals

    r = y; % Initialize the residual
    T = double.empty(0,S); % Initialize the support set
    x_hat_T = zeros(S,C);  % Initialize the sparse signal estimate

    for i = 1:nchoosek(N,S) % Theoretical max you should need

        % Add top S correlated column indice to support set
        [~, J] = maxk(abs(A' * r), S);
        T_i = union(T, J');

        x_hat_T_i = SIGMA_S(pinv(A(:,T_i)) * y, S);
        T_i(~find(x_hat_T_i)) = [];
        x_hat_T_i(~find(x_hat_T_i)) = [];

        % Get residual for best estimate
        r_i = y - A(:,T_i)*x_hat_T_i;


        if  (norm(r_i,2) >= norm(r,2)) || (norm(r, 2) < tol)
            break;
        else

            % Update residual, support set, and sparse x then continue
            r = r_i;
            T = T_i;
            x_hat_T = x_hat_T_i;
        end
    end

    % Build full signal estimate vector
    x_hat = zeros(N,C);
    if ~isempty(T)
        x_hat(T,:) = x_hat_T;
    end
```

```
    % Calculate Measurement error for full estimate
    error  = norm(y-A*x_hat, 2);
end


function mustHaveSameRows(a,b)
% Validate that a and b have the same number of rows
    if ~isequal(size(a,1), size(b,1))
        error('Inputs must be have the same number of rows.');
    end
end


function  v_s = SIGMA_S(v, s)
% SIGMA_S  zeros out all but the highest s values of v
    arguments
        v {mustBeNumeric}
        s {mustBeInteger, mustBePositive}
    end

    [~, idx] = maxk(abs(v),s);
    v_s = zeros(size(v), 'like', v);
    v_s(idx) = v(idx);
end
```

```
function [x_hat, error] = SOLVE_OMP(A, y, S, tol)
% Orthogonal Matching Pursuit (OMP) algorithm to recover a sparse signal.
%
% Inputs:
%   A     - Sensing matrix (M x N)
%   y     - Measurement vector (M x C)
%   S     - Sparsity level (number of non-zero elements in x)
%   tol   - Optional tolerance to stop iterating
%
% Output:
%   x_hat - Reconstructed sparse signal (N x C)
%   error - Error between A*x_hat and y
    arguments
        A   {mustBeNumeric}
        y   {mustBeNumeric, mustHaveSameRows(A,y)}
        S   {mustBeInteger, mustBePositive}
        tol {mustBeReal, mustBePositive} = 1e-15
    end

    % Get dimension of sensing and measures
    [~, N] = size(A);
    [~, C] = size(y);

    r = y; % Initialize the residual
    T = double.empty(0,S); % Initialize the support set
    x_hat_T = zeros(S,C);  % Initialize the sparse signal estimate

    for i = 1:S

        % Add most correlated column indice to support set
        [~, J] = max(abs(A'*r));
        T_i = union(T,J);

        % Solve the lq to get best estimate with the support set
        x_hat_T_i = pinv(A(:,T_i))*y;

        % Get residual for best estimate
        r_i = y - A(:,T_i)*x_hat_T_i;

        if  (norm(r_i,2) >= norm(r,2)) || (norm(r, 2) < tol)
            break;
        else
            % Update residual, support set, and sparse x then continue
            r = r_i;
            T = T_i;
            x_hat_T = x_hat_T_i;
        end
    end

    % Build full signal estimate vector
    x_hat = zeros(N,C);
    if ~isempty(T)
        x_hat(T,:) = x_hat_T;
    end
    % Calculate Measurement error for full estimate
    error  = norm(y-A*x_hat, 2);
```

```
end


function mustHaveSameRows(a,b)
% Validate that a and b have the same number of rows
    if ~isequal(size(a,1), size(b,1))
        error('Inputs must be have the same number of rows.');
    end
end
```

```
function x_hat = SOLVE_L1(A, y)
% SOLVE_L1 recovers a sparse signal using l1-minimization via linprog.

    [~, N] = size(A);
    [~, C] = size(y);

    % Formulate for linprog: min f'*z s.t. Aeq*z = beq, z >= 0
    % Let x = u - v, where u,v >= 0. Then ||x||_1 = sum(u+v).
    % z = [u; v]

    f = ones(2*N, C); % Objective function to minimize sum(u+v)
    Aeq = [A, -A];    % Constraint A(u-v) = y
    beq = y;
    lb = zeros(2*N, C); % Lower bound u,v >= 0

    % Suppress verbose output
    options = optimoptions('linprog', 'Display', 'none');

    % Solve the linear program
    z = linprog(f, [], [], Aeq, beq, lb, [], options);

    if isempty(z)
        x_hat = zeros(N, 1);
        return;
    end

    % Reconstruct x from u and v
    x_hat = z(1:N) - z(N+1:2*N);
end
```